

OBSERVABILITY

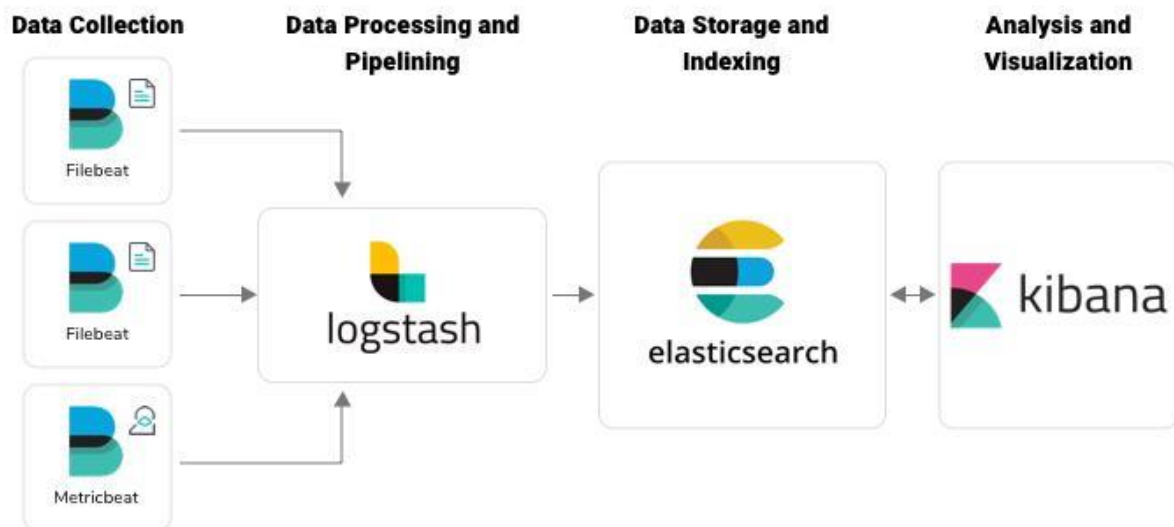
Observability platforms help monitor, troubleshoot, and analyze the health and performance of applications, servers, containers, and cloud infrastructure.

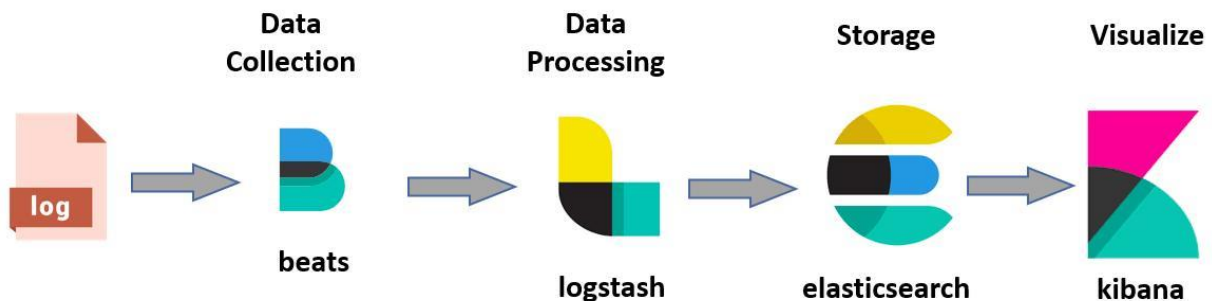
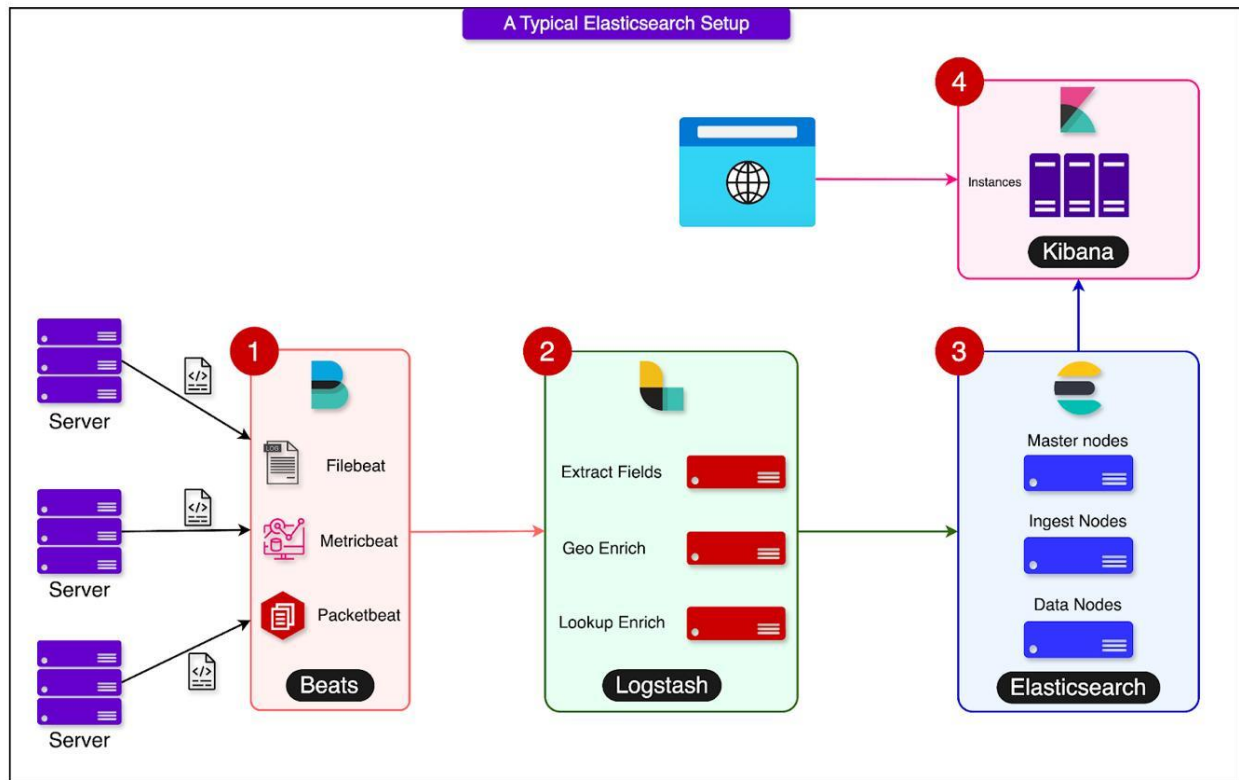
Three platforms are the **ELK Stack**, **Splunk**, and **Grafana**.

Tool	Primary Purpose	Data Collected	Best For
ELK Stack	Log management and analytics	Logs	Centralized log analysis
Splunk	Enterprise observability and security analytics	Logs, metrics, traces	Large-scale enterprise monitoring
Grafana	Visualization and observability	Metrics, logs, traces	Dashboards and real-time monitoring

1. ELK Stack (Elastic Stack)

The ELK Stack is an open-source platform used to collect, store, search, analyze, and visualize log data.





Components

- Elasticsearch**

- Distributed search and analytics engine
- Stores indexed data
- Performs fast searches

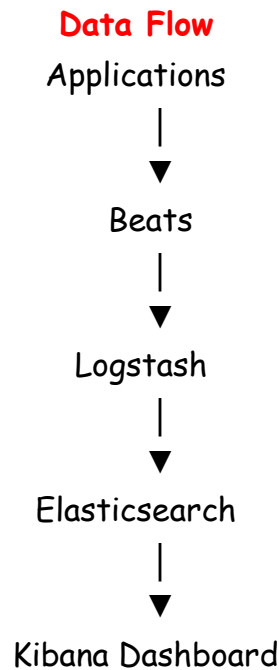
- Logstash**

- Data collection and processing pipeline
- Parses and transforms logs
- Supports hundreds of input sources

- Kibana**

- Visualization and dashboard tool

- Creates graphs and reports
- Performs log searches
- **Beats**
 - Lightweight data shippers
 - Examples:
 - Filebeat (logs)
 - Metricbeat (system metrics)
 - Packetbeat (network traffic)
 - Heartbeat (uptime monitoring)



Features

- Centralized logging
- Full-text search
- Log correlation
- Dashboard creation
- Alerting
- Machine learning (licensed editions)

Advantages

- Open source
- Highly scalable
- Fast search

- Rich visualization
- Large community

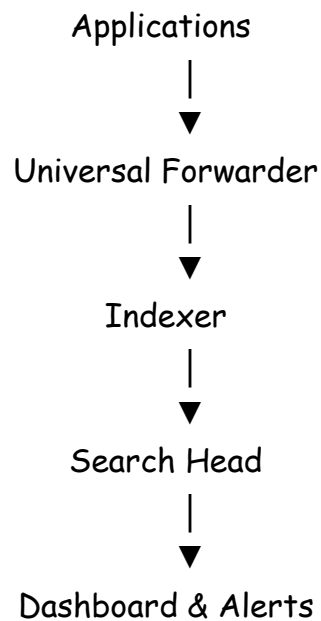
Limitations

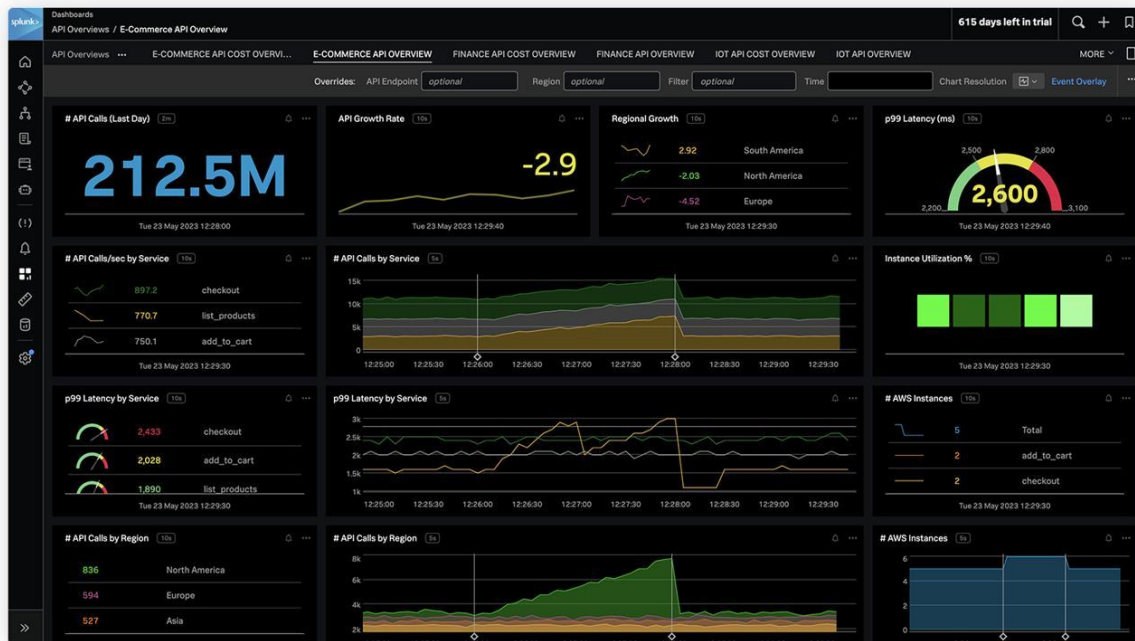
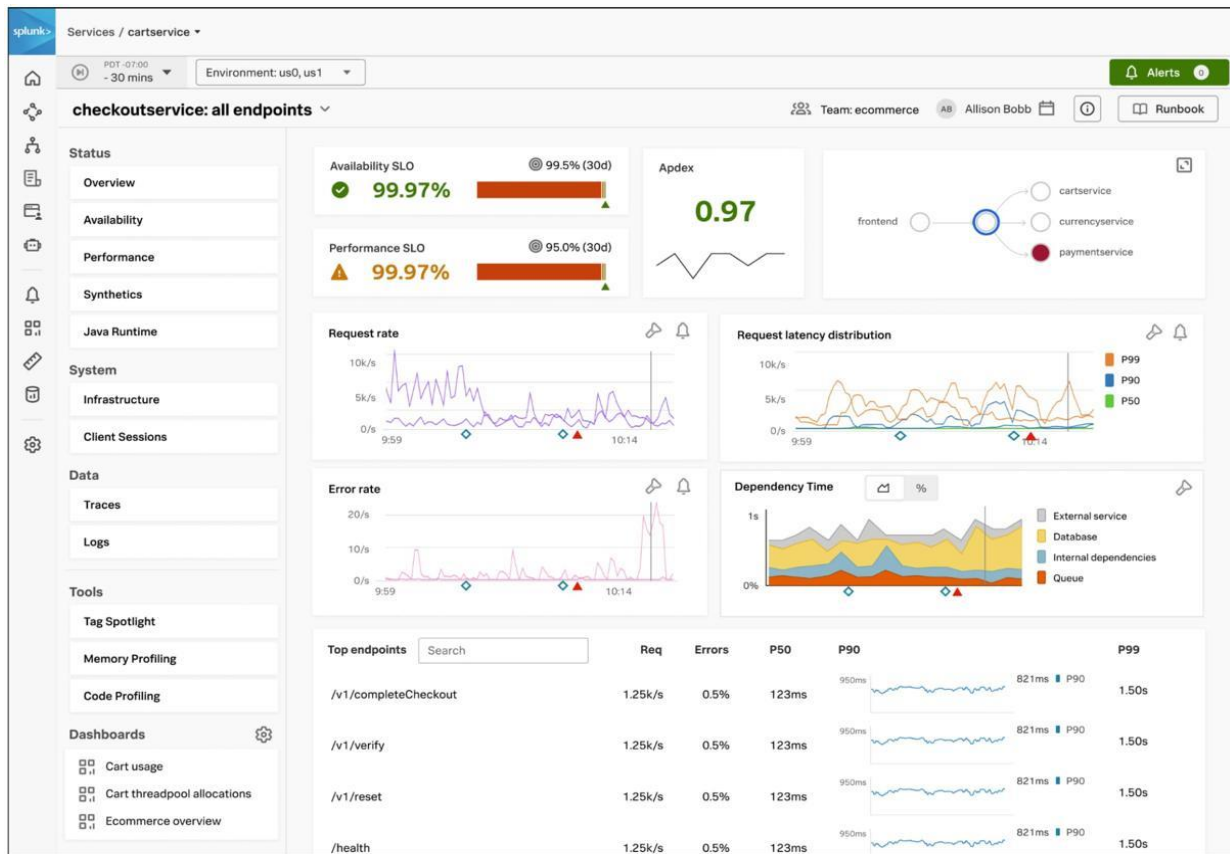
- Requires operational management
 - Memory intensive
 - Advanced features may require commercial licensing
-

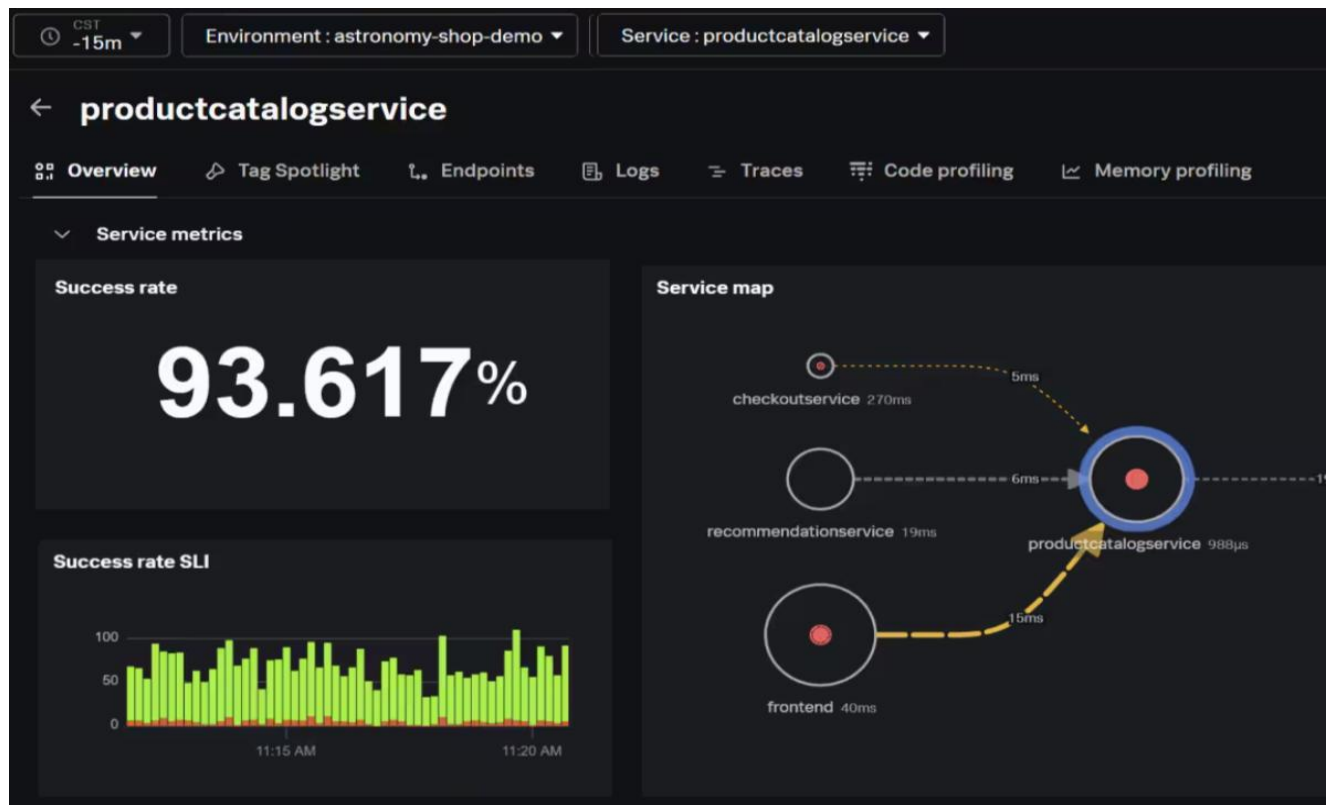
2. Splunk

Splunk is an enterprise platform for searching, monitoring, and analyzing machine-generated data.

Architecture







Features

- Real-time log analysis
- Metrics monitoring
- Distributed tracing
- AI-assisted anomaly detection
- Security monitoring (SIEM)
- IT operations analytics
- Alerting
- Report generation

Common Use Cases

- Application monitoring
- Infrastructure monitoring
- Security incident detection
- Compliance reporting
- Business analytics

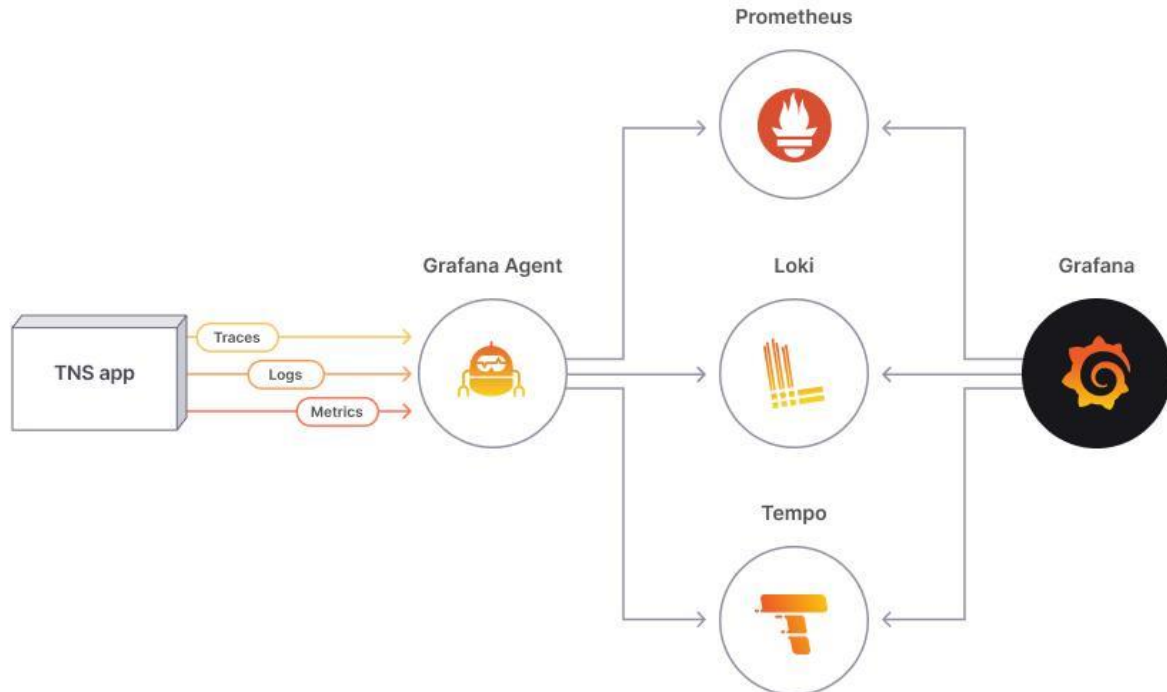
Advantages

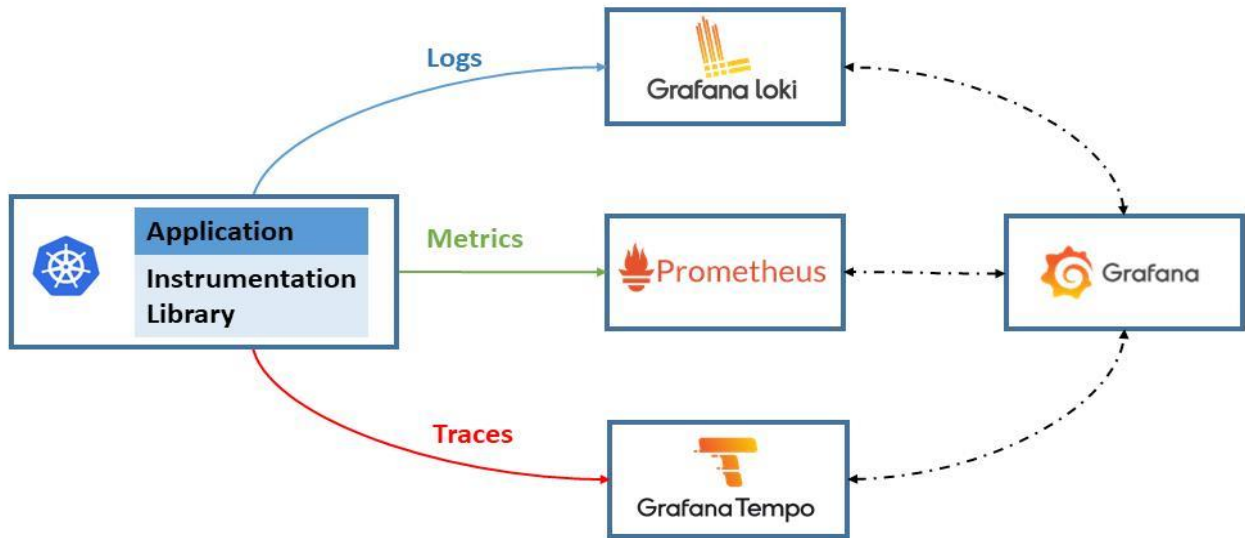
- Enterprise-ready
- Powerful search language (SPL)
- Excellent support
- Scalable architecture
- Advanced analytics

Limitations

- Commercial licensing can be expensive
- Learning SPL takes time
- Higher infrastructure requirements

3. Grafana



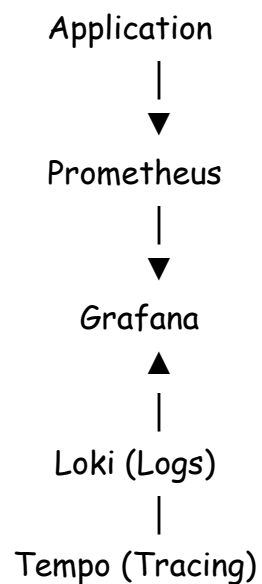


Grafana is an open-source visualization and observability platform that integrates with many data sources.

Grafana Ecosystem

- **Grafana** - Dashboards and visualization
- **Prometheus** - Metrics collection
- **Loki** - Log aggregation
- **Tempo** - Distributed tracing

Architecture



Features

- Interactive dashboards
- Real-time monitoring
- Alerts
- Multiple data sources
- Plugin ecosystem
- Infrastructure monitoring
- Kubernetes monitoring

Advantages

- Open source
- Excellent dashboards
- Lightweight
- Easy integration
- Strong Kubernetes support

Limitations

- Requires external storage backends for logs and metrics
- Some advanced enterprise capabilities are paid

Comparison

Feature	ELK Stack	Splunk	Grafana
Open Source	Yes (core)	No	Yes (core)
Log Management	Excellent	Excellent	Via Loki
Metrics	Good	Excellent	Excellent
Distributed Tracing	Yes	Yes	Via Tempo
Dashboards	Kibana	Splunk Dashboard	Excellent
Alerting	Yes	Yes	Yes
AI/ML	Available	Advanced	Limited

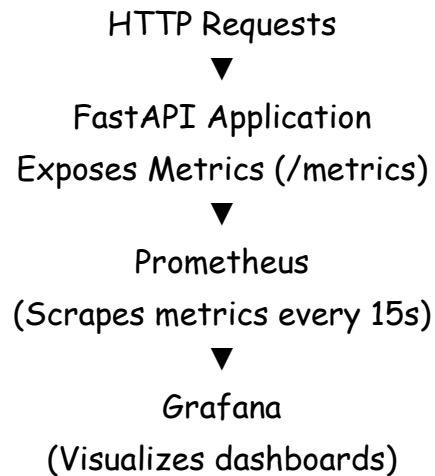
Feature	ELK Stack	Splunk	Grafana
Cost	Free + commercial options	Commercial	Free + commercial options
Learning Curve	Medium	High	Easy to Medium
Best For	Centralized logging	Enterprise observability and security	Visualization and cloud-native monitoring

When to Use Each

- **ELK Stack**
 - Best when your primary need is centralized log management and search.
 - Suitable for organizations wanting an open-source solution.
- **Splunk**
 - Best for large enterprises requiring comprehensive observability, advanced analytics, and security (SIEM) capabilities with commercial support.
- **Grafana**
 - Best for cloud-native environments, Kubernetes, and organizations focused on metrics visualization, dashboards, and integrating logs and traces through tools like Prometheus, Loki, and Tempo.

Working with Grafana and Fast API

Use **Grafana** with **FastAPI** is to expose application metrics using **Prometheus**, collect them with Prometheus, and visualize them in Grafana.



Demo: d:\advfastapi

FastAPI + Prometheus + Grafana (Without Docker)

Step 1: Create the Project Folder

Create a folder → advfastapi

Project structure

```
advfastapi/
|
├── app.py
├── requirements.txt
└── prometheus.yml
```

Step 2: Create Virtual Environment (Recommended)

Open Command Prompt.

```
cd fastapi-grafana-demo
```

Create virtual environment

```
python -m venv env
```

Activate it → `env\Scripts\activate`

Step 3: Install Required Packages

```
pip install fastapi uvicorn prometheus-fastapi-instrumentator  
pip freeze > requirements.txt
```

Step 4: Create FastAPI Application

Create `app.py`

```
from fastapi import FastAPI  
from prometheus_fastapi_instrumentator import Instrumentator  
import random
```

```
app = FastAPI()
```

```
Instrumentator().instrument(app).expose(app)
```

```
@app.get("/")  
def home():  
    return {"message": "Welcome"}
```

```
@app.get("/users")  
def users():
```

```
return {"users": random.randint(10,100)}
```

```
@app.get("/orders")
def orders():
    return {"orders": random.randint(100,500)}
```

Step 5: Run FastAPI

uvicorn app:app --reload

You should see :-->INFO: Uvicorn running on http://127.0.0.1:8000

Open browser → http://localhost:8000

Metrics endpoint: → http://localhost:8000/metrics

You should see Prometheus metrics like

```
http_requests_total
python_gc_objects_collected_total
process_cpu_seconds_total
...
```

Leave this terminal running.

Step 6: Install Prometheus

Download the Windows ZIP from the official Prometheus site:

- [Prometheus Downloads](#)

Example folder

C:\prometheus

Contents

prometheus.exe
promtool.exe
prometheus.yml (Replace with your code)

Step 7: Replace prometheus.yml

Copy this into

C:\prometheus\prometheus.yml

global:

scrape_interval: 5s

scrape_configs:

- job_name: "fastapi"

static_configs:

- targets:

- 127.0.0.1:8000

Notice that we use :127.0.0.1:8000

Step 8: Start Prometheus

Open another Command Prompt.

Copy zipfile content into advfastapi project folder.

Copy prometheus.yml into Prometheus folder

Run

prometheus.exe --config.file=prometheus.yml

Open

http://localhost:9090

Click

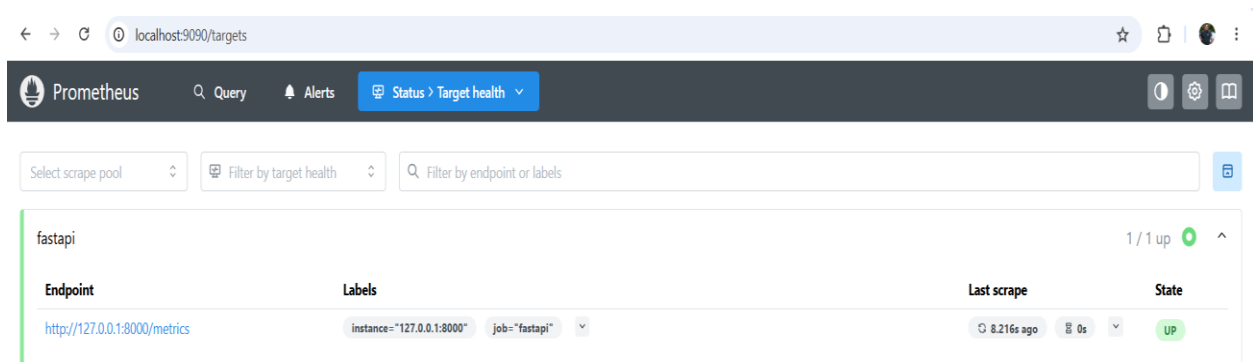
Status

→ Targets

You should see

fastapi

UP



If it says **UP**, Prometheus is successfully scraping your FastAPI metrics.

Step 9: Install Grafana

Download the Windows installer from the official Grafana site:

- [Grafana Downloads](#)
-
- **Standalone Windows Binaries**(64 Bit)SHA256: 378c8cc9d8bf82d59e3e79abd20ea3a09015ab8be187bf0fe669fdf2d2f04c3d
- [Download the zip file](#) (grafana-enterprise_13.1.1_29761037902_windows_amd64.tar.gz) and follow the instructions in the installation guide below.
-

Install the standalone binary file xyz.tar.gz file and paste in advfastapi folder.

Step 10: Start Grafana (it is written in typescript)

Goto this folder and run grafanna.exe file:

D:\advfastapi\grafana-
enterprise_13.1.1_29761037902_windows_arm64.tar\grafana-13.1.1\bin

<grafana server

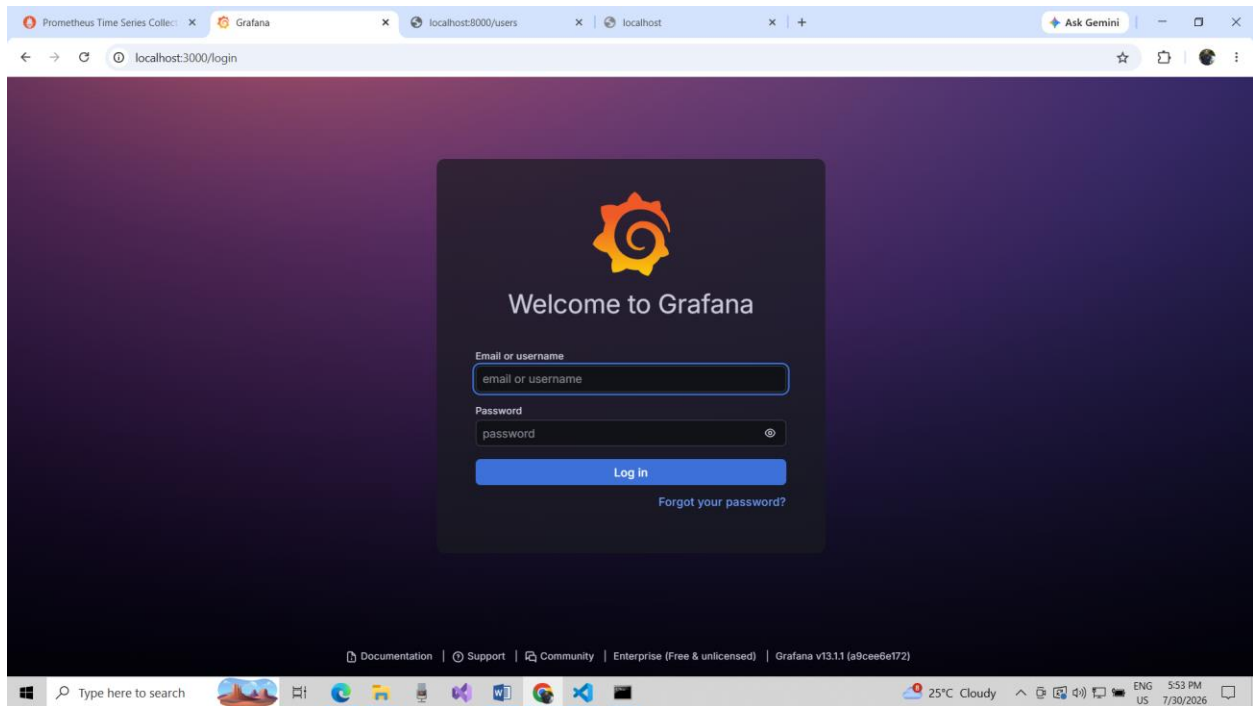
Or

➤ grafana server --homepath "D:\advfastapi\grafana-13.1.1"

Wait till all scrolling messages stops Once plugins are installed then

Open

<http://localhost:3000>

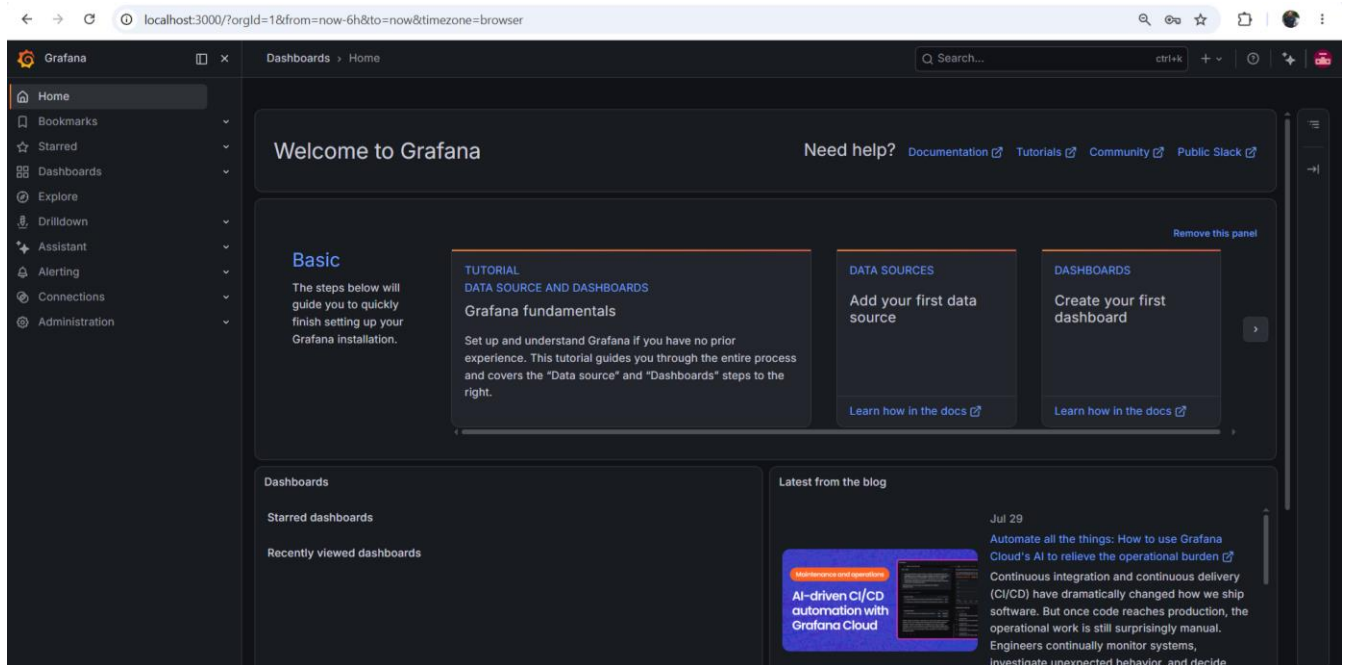


Login

Username: admin

Password: admin

Grafana may ask you to change the password on first login. (SKIP)



Step 11: Add Prometheus Data Source

Go to

Connections → Click Add New Connection button (connect to datasource)

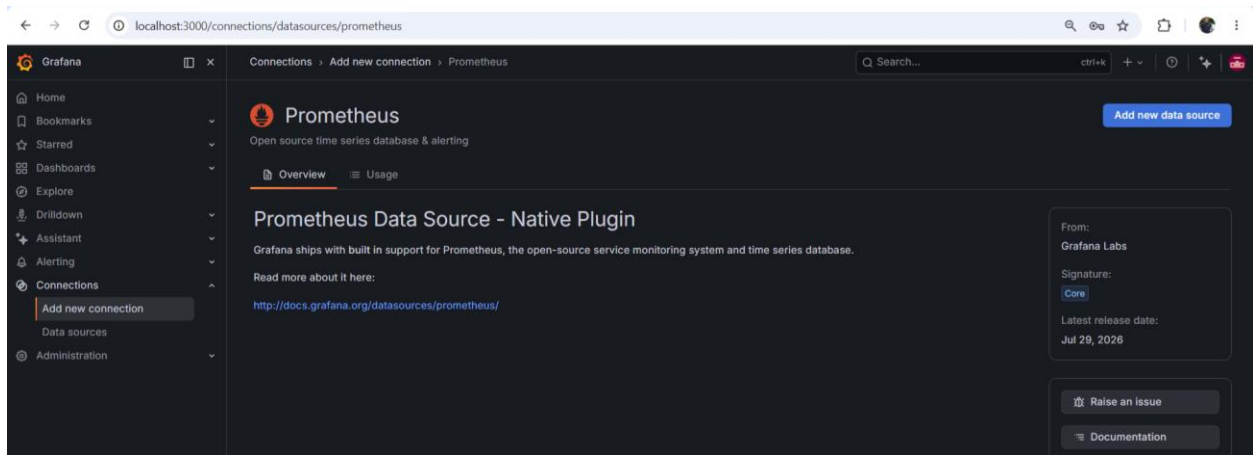
→ Data Sources

→ Add data source

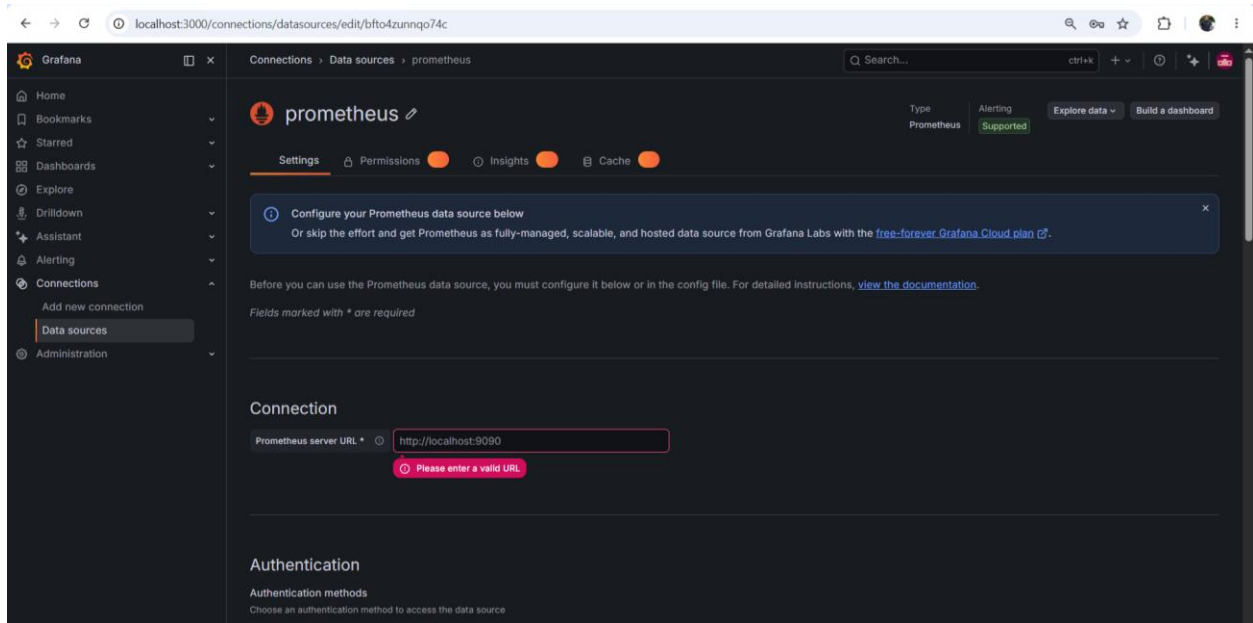
Choose → Prometheus → URL → <http://localhost:9090>

Click → Save & Test

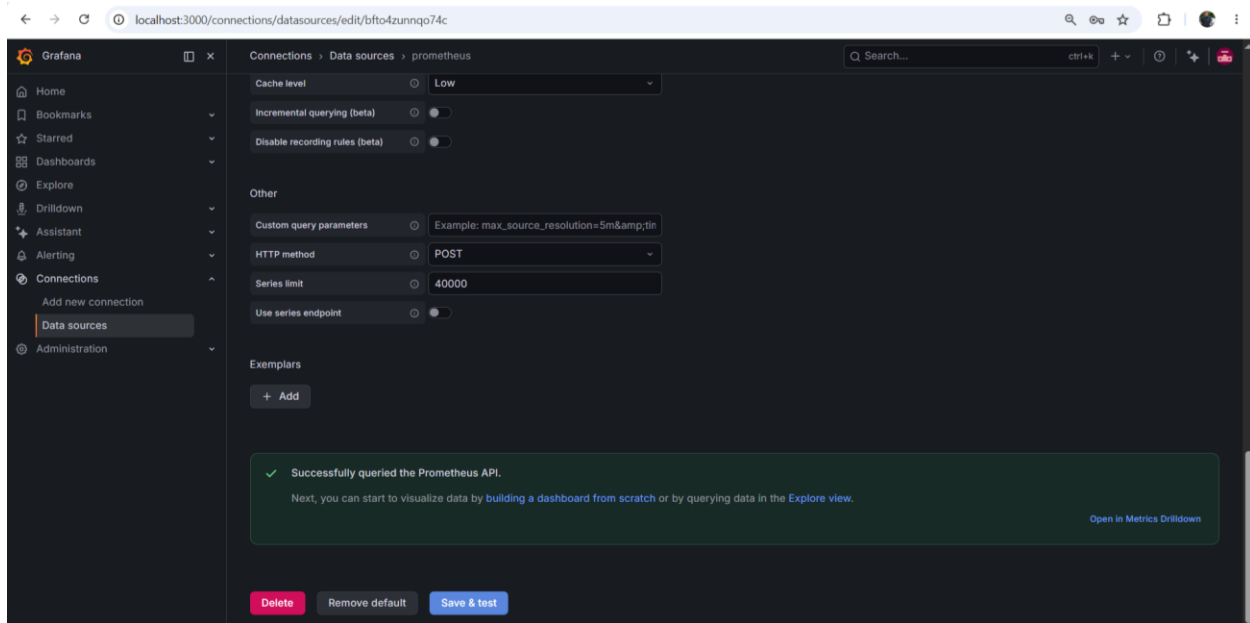
Expected message → Data source is working



Click Add Data Source:

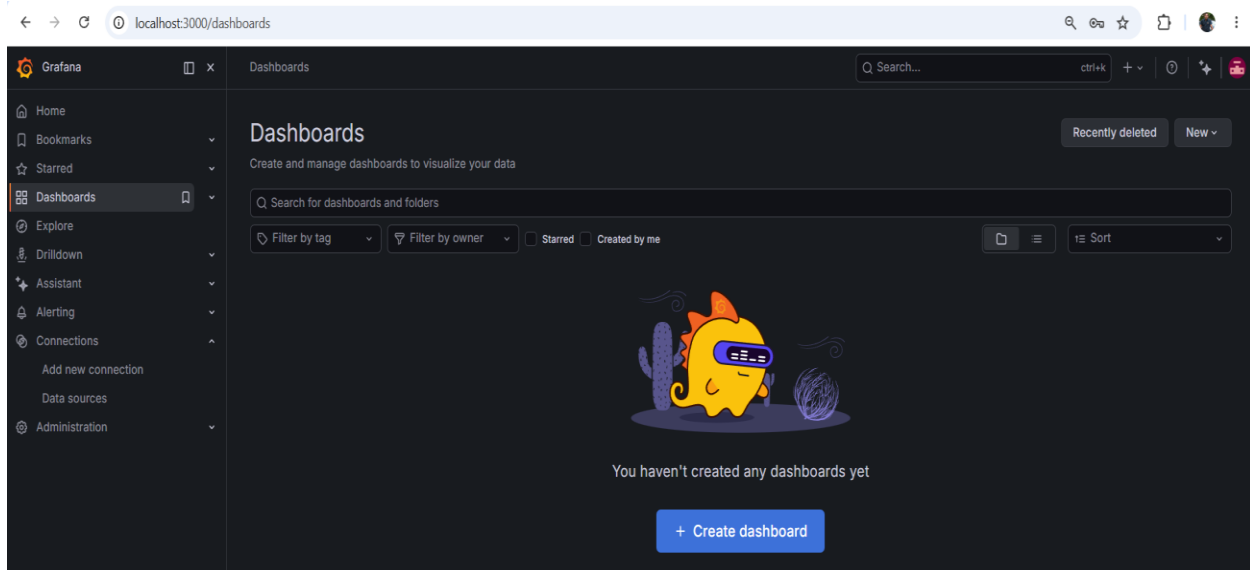


Scroll down by leaving everything as it is and click save and test:

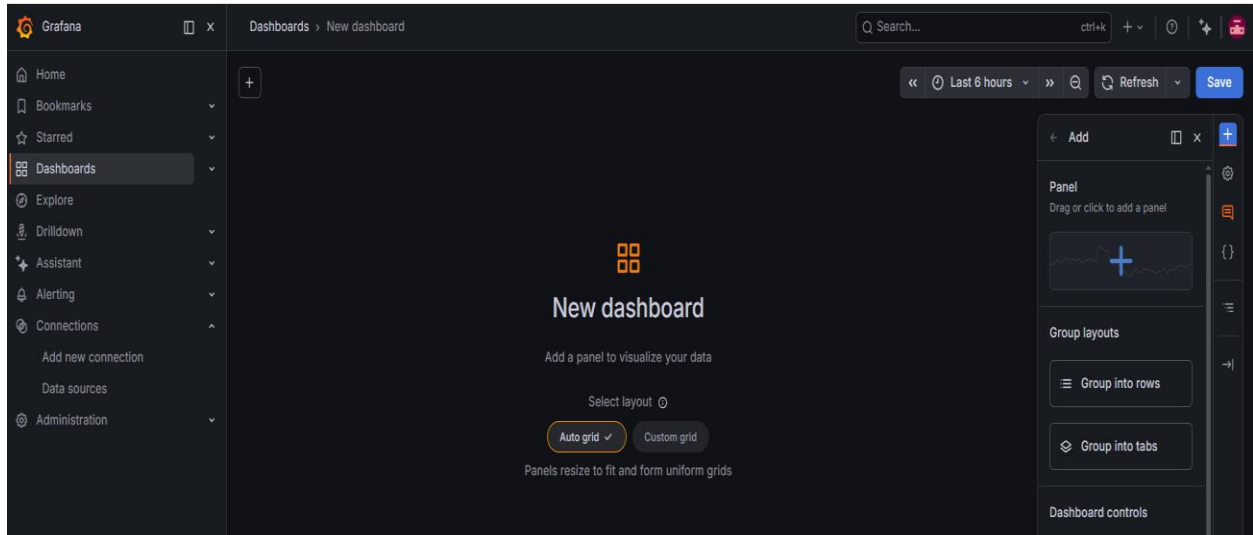


Step 12: Create Dashboard

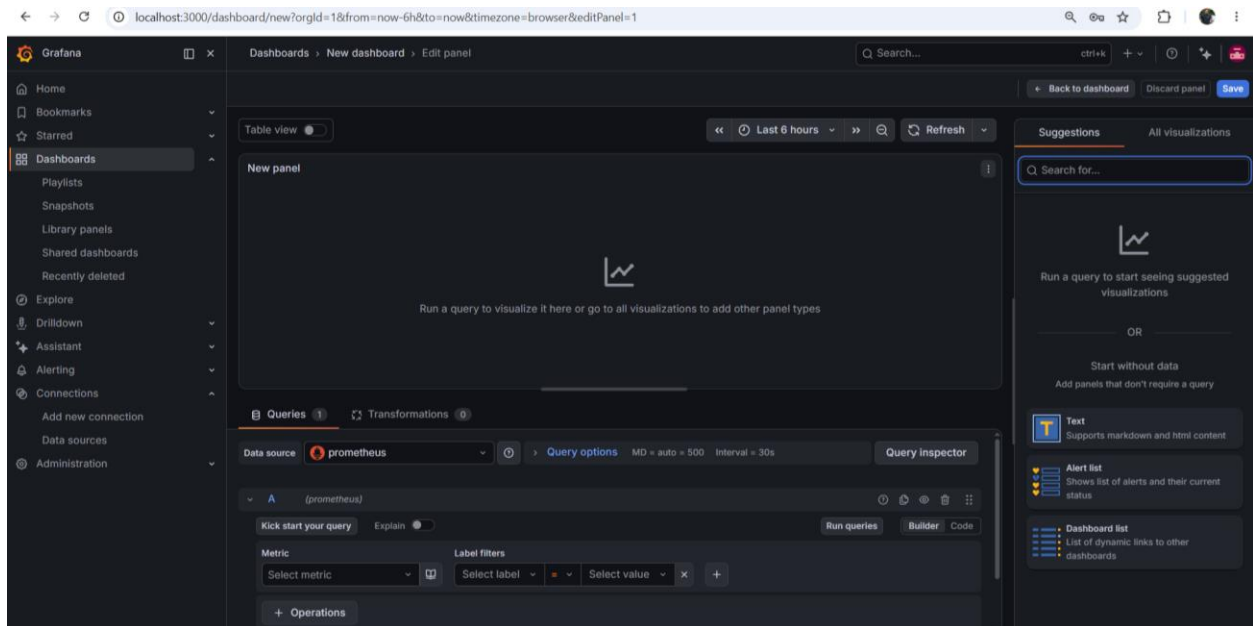
Dashboards → New Dashboard → New Panel → create visualization



Choose Autogrid:



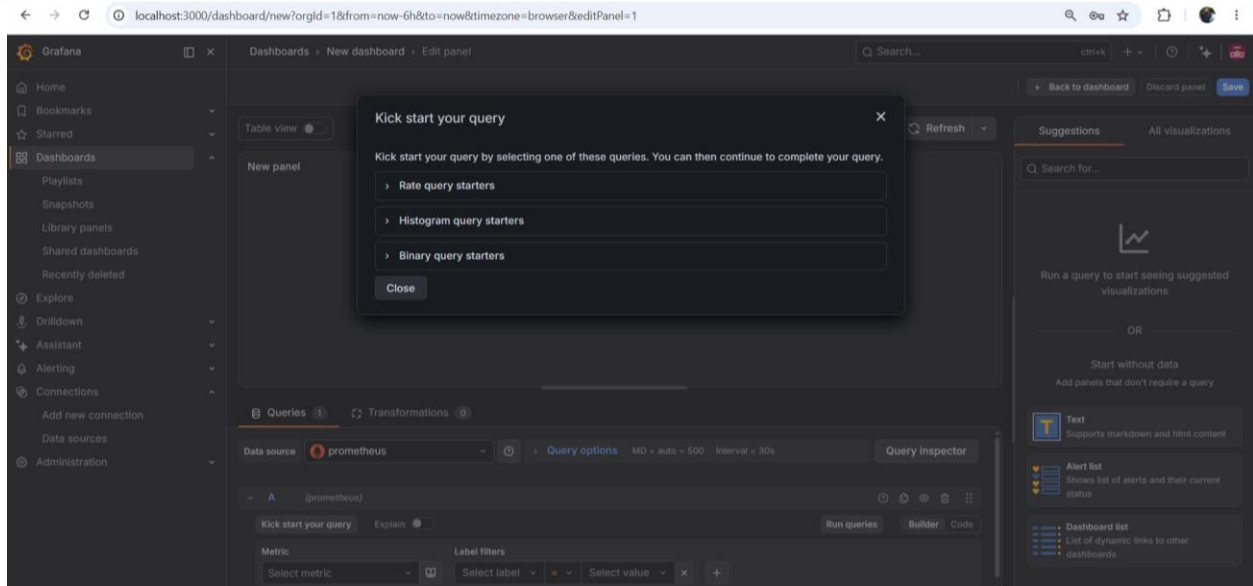
In panel section right side, Click + symbol add create visualization. Observe below screen. Choose Data source as Prometheus.



Choose Prometheus

Step 13: Useful Queries

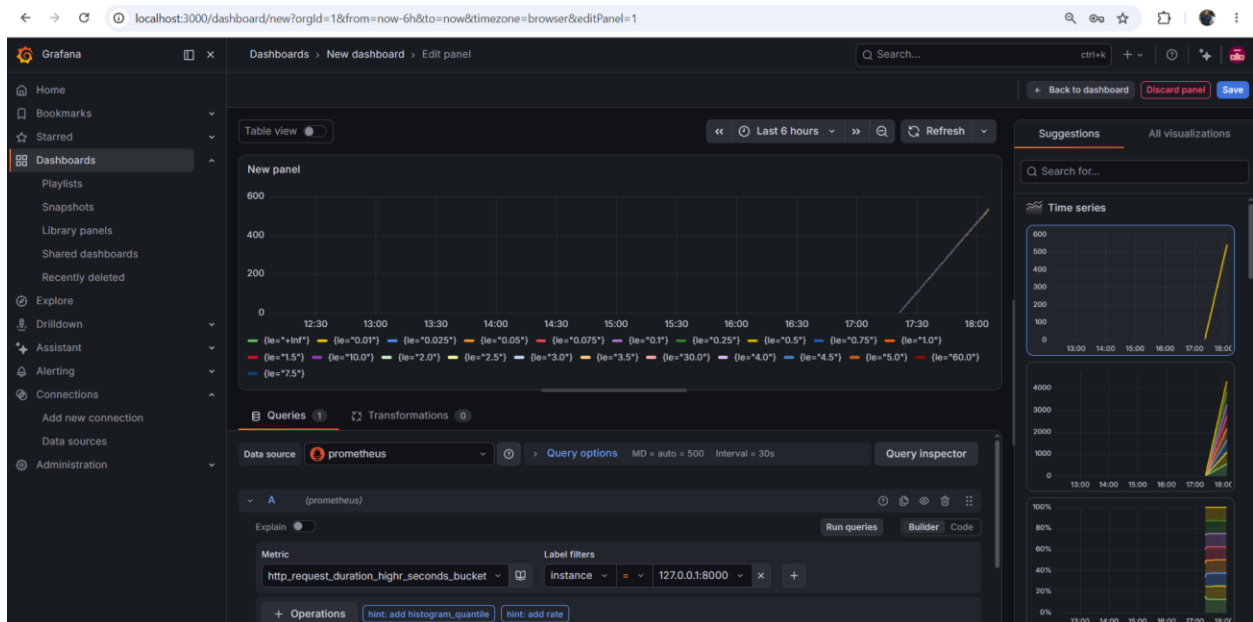
Click start your queries:



From dropdown select : `http_request_duration_higher_seconds_bucket`

From label : select instance

url: `127.0.0.1:8000`

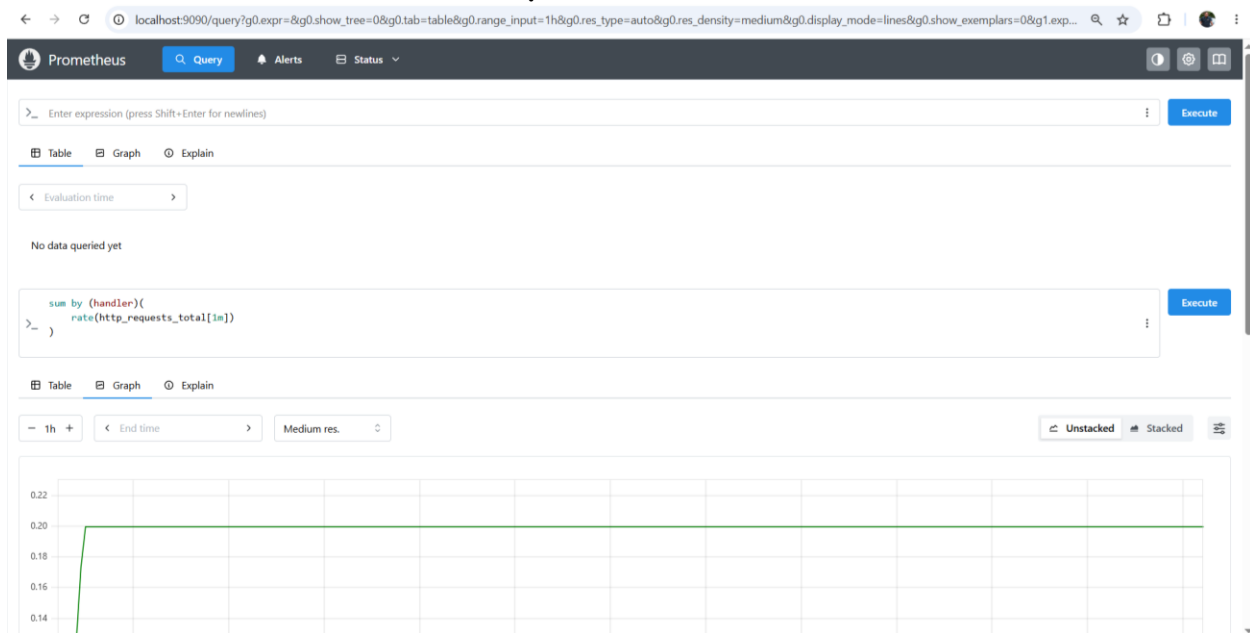


Goto Prometheus browser tab and type query : sum by (handler)(

rate(http_requests_total[1m])

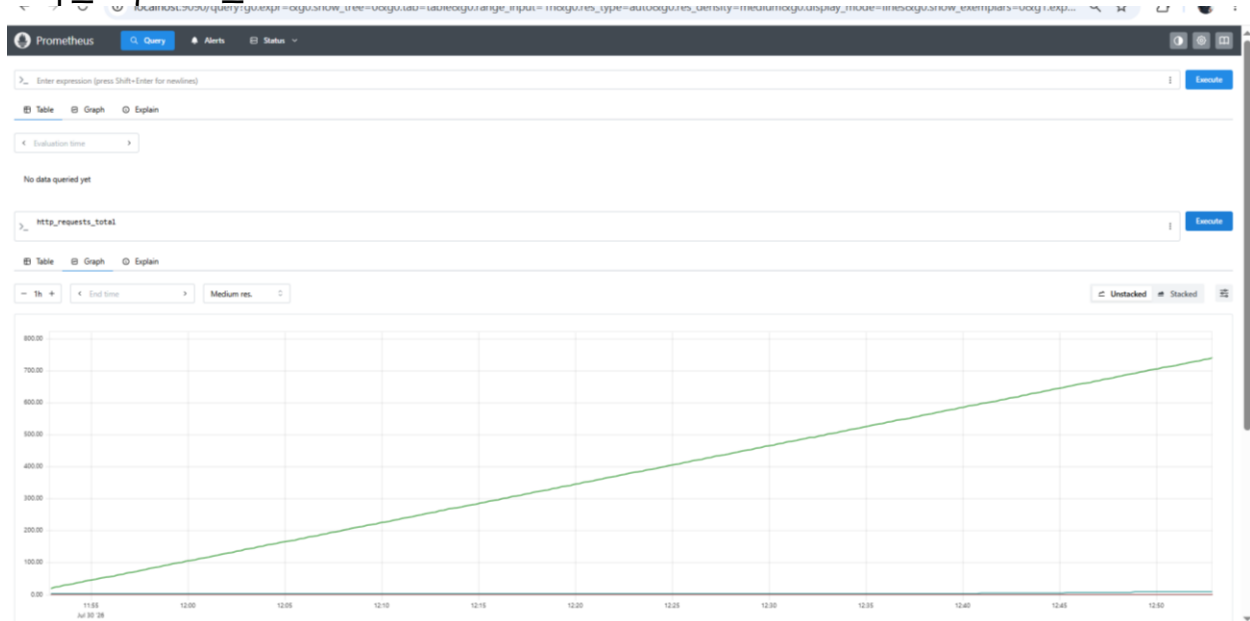
)

And click **execute** to see this output.



Total Requests

`http_requests_total`



Request Rate

```
rate(http_requests_total[1m])
```

Average Response Time

```
rate(http_request_duration_seconds_sum[1m])  
/  
rate(http_request_duration_seconds_count[1m])
```

Requests per Endpoint

```
sum by (handler)(  
    rate(http_requests_total[1m])  
)
```

CPU Usage

```
process_cpu_seconds_total
```

Memory Usage

```
process_resident_memory_bytes
```

Python Garbage Collection

```
python_gc_objects_collected_total
```

Step 14: Generate Traffic (Click Graph to see the output)

Open these URLs several times:

```
http://localhost:8000/
```

`http://localhost:8000/users`
`http://localhost:8000/orders`

Or refresh them repeatedly to generate requests.

Step 15: Refresh Grafana

Return to your Grafana dashboard and refresh.

You should see graphs updating in real time as requests are made.

Running Everything (Three Terminals)

Keep these running simultaneously:

Terminal 1 (FastAPI)

`uvicorn app:app --reload`

Terminal 2 (Prometheus)

`cd C:\prometheus`
`prometheus.exe --config.file=prometheus.yml`

Terminal 3 (Grafana)

Grafana

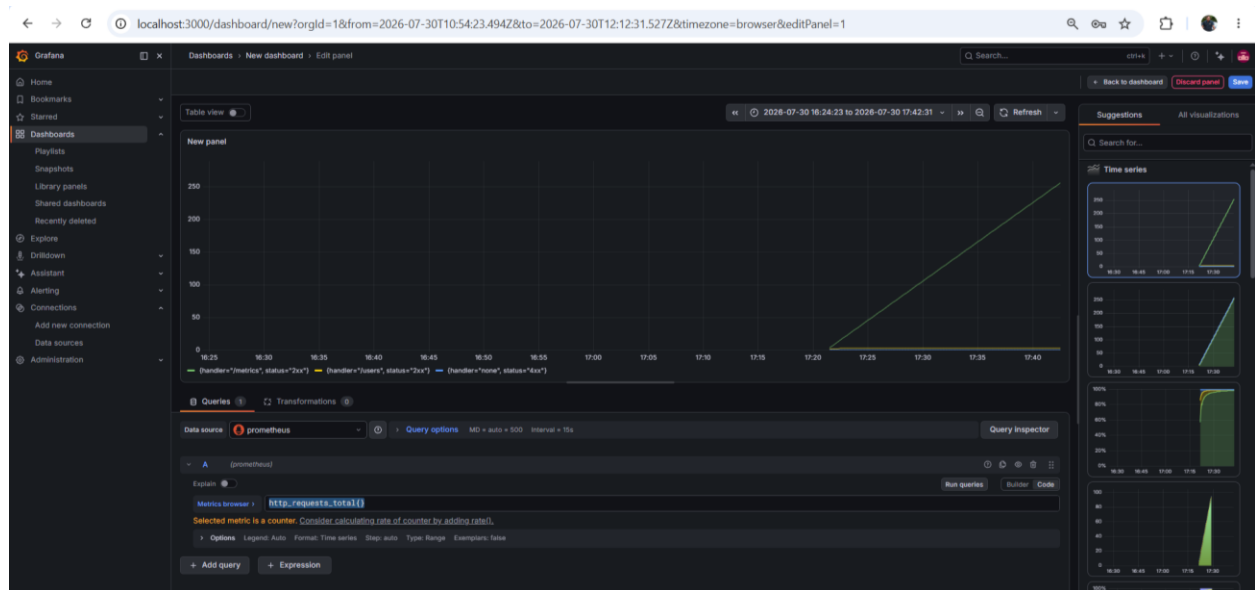
Run `grafana.exe`

Application URLs

Component	URL
FastAPI	http://localhost:8000
Metrics	http://localhost:8000/metrics

Component	URL
Prometheus	http://localhost:9090
Grafana	http://localhost:3000

This setup provides the same functionality as the Docker-based version but uses native Windows installations for Prometheus and Grafana.



Steps with Docker:

Step 1: Install Docker

Create a folder

fastapi-grafana-demo

Project structure

fastapi-grafana-demo/

|

|— app.py

|— requirements.txt

- └─ prometheus.yml
- └─ docker-compose.yml

Step 2: Install Python Packages

```
pip install fastapi uvicorn prometheus-fastapi-instrumentator
```

Step 3: Create FastAPI Application

app.py

```
from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator
import random
```

```
app = FastAPI()
```

```
Instrumentator().instrument(app).expose(app)
```

```
@app.get("/")
def home():
    return {"message": "Welcome"}
```

```
@app.get("/users")
def users():
    return {
        "users": random.randint(10,100)
    }
```

```
@app.get("/orders")
def orders():
    return {
        "orders": random.randint(100,500)
    }
```

Run

```
uvicorn app:app --reload
```

Open

```
http://localhost:8000
```

Metrics endpoint

```
http://localhost:8000/metrics
```

You should see

```
http_requests_total
process_cpu_seconds_total
python_gc_objects_collected_total
...
```

Step 4: Configure Prometheus

Create

prometheus.yml

```
global:
```

```
  scrape_interval: 5s
```

```
scrape_configs:
```

```
- job_name: "fastapi"
```

```
  static_configs:
```

```
    - targets:
```

```
      - host.docker.internal:8000
```

Step 5: Docker Compose

Create

docker-compose.yml

version: "3"

services:

prometheus:

image: prom/prometheus

container_name: prometheus

ports:

- "9090:9090"

volumes:

- ./prometheus.yml:/etc/prometheus/prometheus.yml

grafana:

image: grafana/grafana

container_name: grafana

ports:

- "3000:3000"

environment:

- GF_SECURITY_ADMIN_USER=admin

- GF_SECURITY_ADMIN_PASSWORD=admin

depends_on:

- prometheus

Run

docker compose up

Step 6: Open Grafana

Open

<http://localhost:3000>

Login

Username: admin

Password: admin

Step 7: Add Prometheus Data Source

Go to

Connections

→ Data Sources

→ Add Data Source

→ Prometheus

URL

http://prometheus:9090

Click

Save & Test

Step 8: Create Dashboard

Create

Dashboard

→ Add Visualization

Choose Prometheus

Example Query 1

Total Requests

http_requests_total

Example Query 2

Request Rate

rate(http_requests_total[1m])

Example Query 3

Average Response Time

```
rate(http_request_duration_seconds_sum[1m])  
/  
rate(http_request_duration_seconds_count[1m])
```

Example Query 4

Requests Per Endpoint

```
sum by (handler)(  
    rate(http_requests_total[1m])  
)
```

Step 9: Generate Traffic

Open several times

`http://localhost:8000/`

`http://localhost:8000/users`

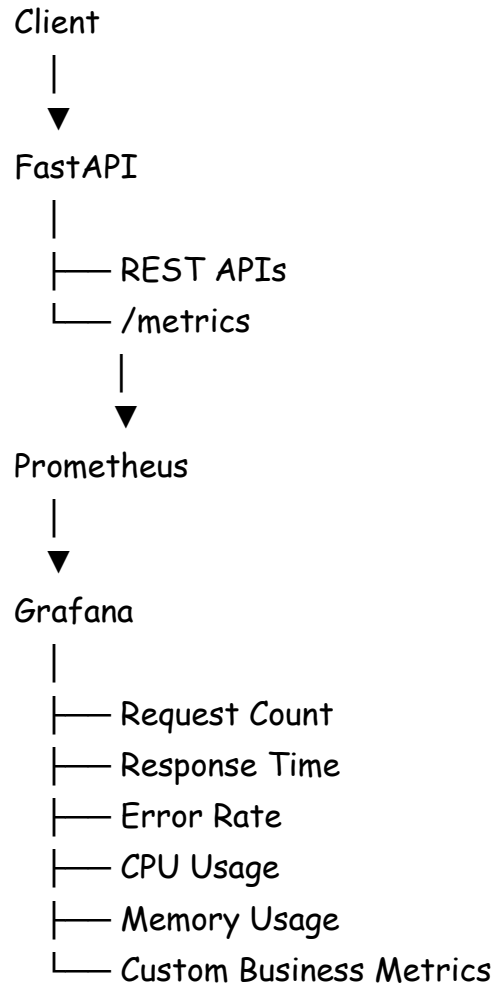
`http://localhost:8000/orders`

Refresh Grafana to see metrics updating.

Step 10: What You Can Monitor

Metric	PromQL
Total Requests	<code>http_requests_total</code>
Request Rate	<code>rate(http_requests_total[1m])</code>
Error Rate	<code>rate(http_requests_total{status="500"}[5m])</code>
Response Time	<code>http_request_duration_seconds</code>
CPU Usage	<code>process_cpu_seconds_total</code>
Memory Usage	<code>process_resident_memory_bytes</code>
Python GC	<code>python_gc_objects_collected_total</code>
Custom Business Metrics	<code>orders_created_total</code>

Expected Flow



This is complete observability pipeline: **FastAPI** exposes metrics, **Prometheus** scrapes them at regular intervals, and **Grafana** provides dashboards to monitor application health, performance, and custom business metrics in real time.
